Digital Design

FSM Optimization

A. Sahu
Dept of Comp. Sc. & Engg.
Indian Institute of Technology Guwahati
http://jatinga.iitg.ac.in/~asahu/cs221/

Outline

- FSM Optimization: **State Minimization**
  - Row Matching method
  - **Partitioning method**
  - **Implication Chart**
- FSM Optimization : State Encoding
  - Binary/Sequential, Gray, One-hot
  - Heuristic Based

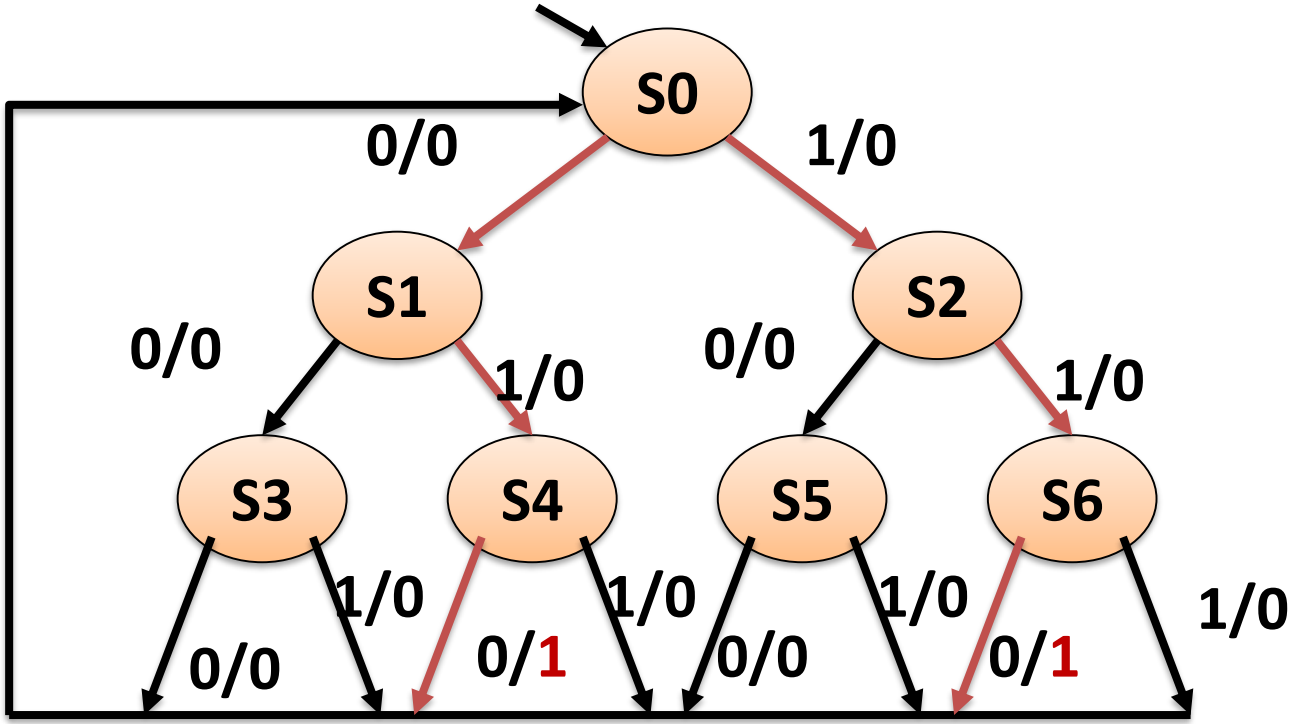
Partitioning Methods

State Minimization: Partitioning

- Form an initial partition (P_1) : that includes **all the** states.
- Form a 2^{nd} partition (P_2)
  - By separating the states into two blocks based upon their output values.
- Form a third partition (P_3)
  - by separating the states into blocks corresponding to the next state values.
- Continue partitioning until
  - Two successive partitions are the same (i.e. $P_{N-1} = P_N$).
- All states in any one block are equivalent
  - Equivalent states can be combined into a single state.

State Minimization: Previous Example

- Sequence Detector for 010 or 110
- After is asserted after each 3 bit input sequence if it consist of 110 or 010



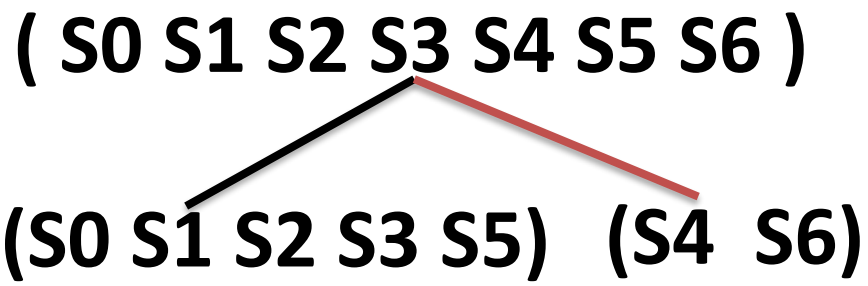
State Minimization Example

Input	PS	NS		OUTPUT	
		X=0	X=1	X=0	X=1
Reset	S0	S1	S2	0	0
0	S1	S3	S4	0	0
1	S2	S5	S6	0	0
00	S3	S0	S0	0	0
01	S4	S0	S0	1	0
10	S5	S0	S0	0	0
11	S6	S0	S0	1	0

(S0 S1 S2 S3 S4 S5 S6)

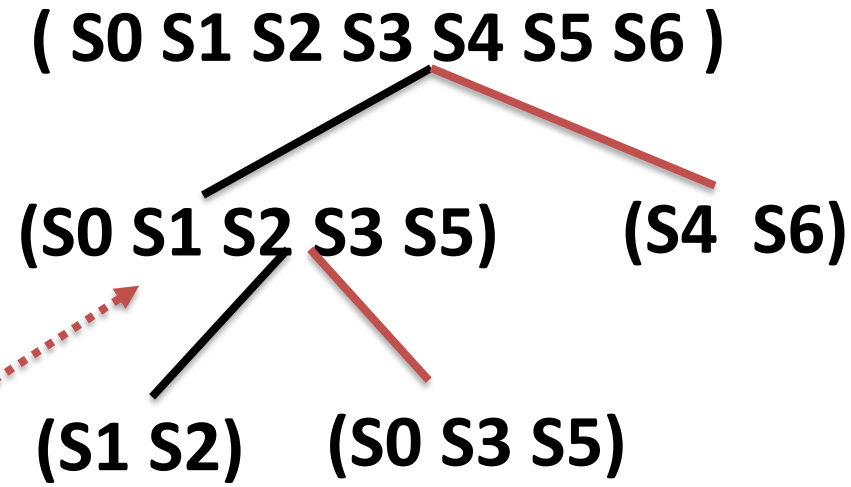
State Minimization Example

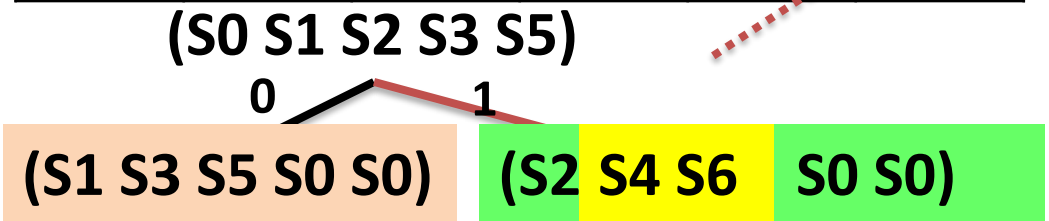
Input	PS	NS		OUTPUT	
		X=0	X=1	X=0	X=1
Reset	S0	S1	S2	0	0
0	S1	S3	S4	0	0
1	S2	S5	S6	0	0
00	S3	S0	S0	0	0
01	S4	S0	S0	1	0
10	S5	S0	S0	0	0
11	S6	S0	S0	1	0



State Minimization Example

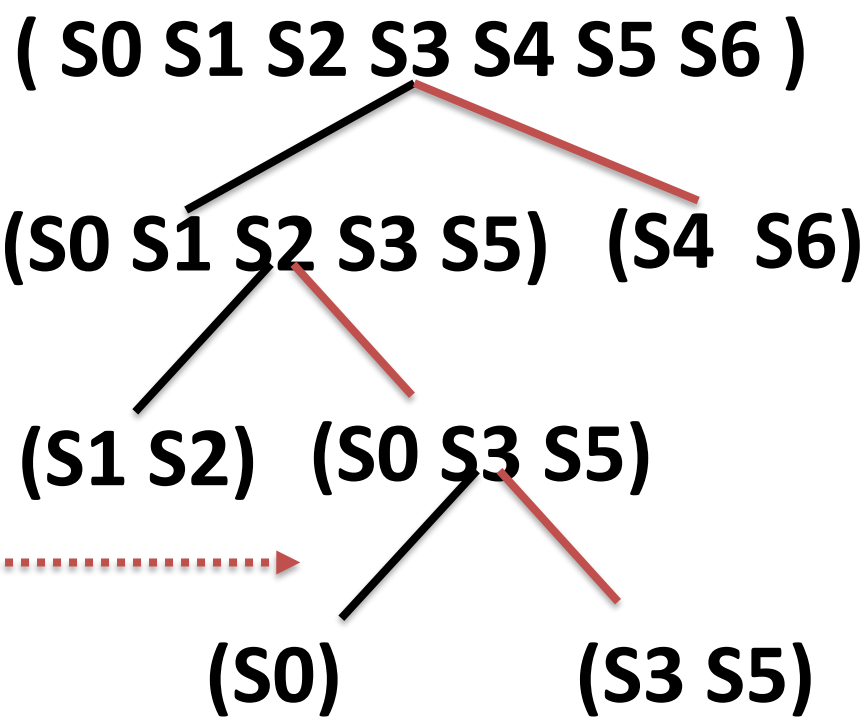
Input	PS	NS		OUTPUT	
		X=0	X=1	X=0	X=1
Reset	S0	S1	S2	0	0
0	S1	S3	S4	0	0
1	S2	S5	S6	0	0
00	S3	S0	S0	0	0
01	S4	S0	S0	1	0
10	S5	S0	S0	0	0
11	S6	S0	S0	1	0





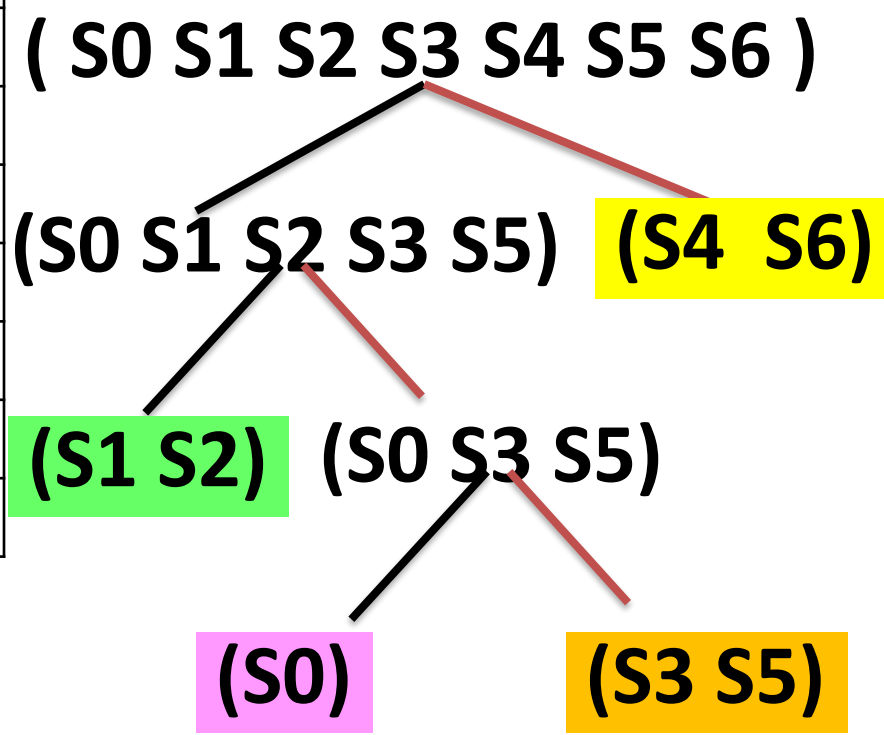
State Minimization Example

Input	PS	NS		OUTPUT	
		X=0	X=1	X=0	X=1
Reset	S0	S1	S2	0	0
0	S1	S3	S4	0	0
1	S2	S5	S6	0	0
00	S3	S0	S0	0	0
01	S4	S0	S0	1	0
10	S5	S0	S0	0	0
11	S6	S0	S0	1	0



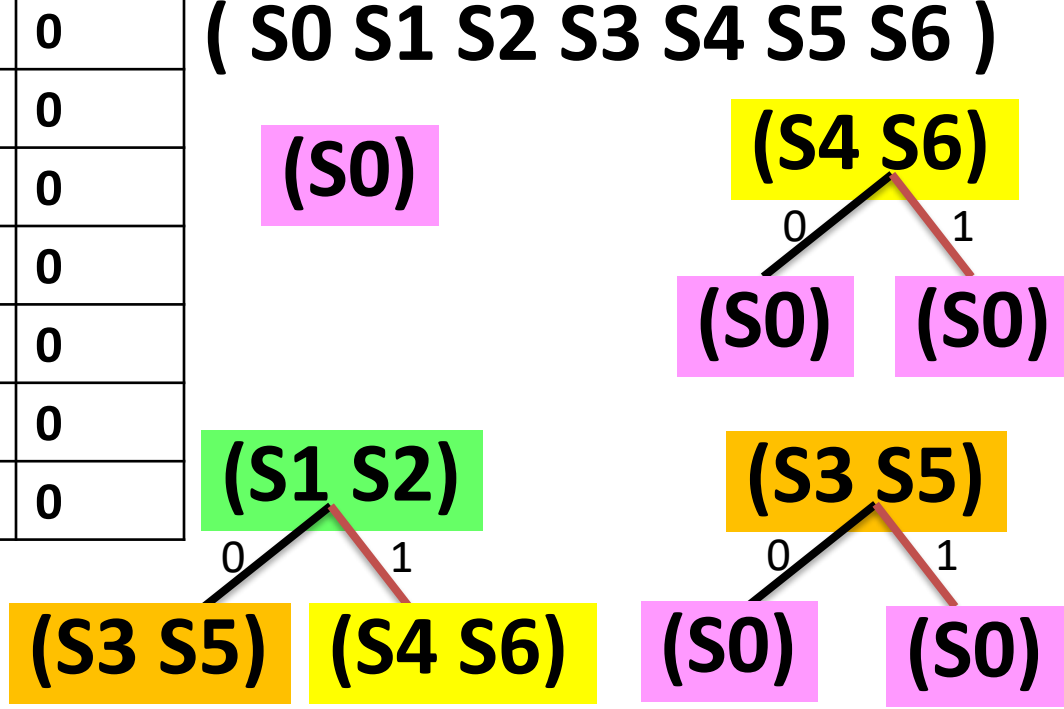
State Minimization Example

Input	PS	NS		OUTPUT	
		X=0	X=1	X=0	X=1
Reset	S0	S1	S2	0	0
0	S1	S3	S4	0	0
1	S2	S5	S6	0	0
00	S3	S0	S0	0	0
01	S4	S0	S0	1	0
10	S5	S0	S0	0	0
11	S6	S0	S0	1	0



State Minimization Example

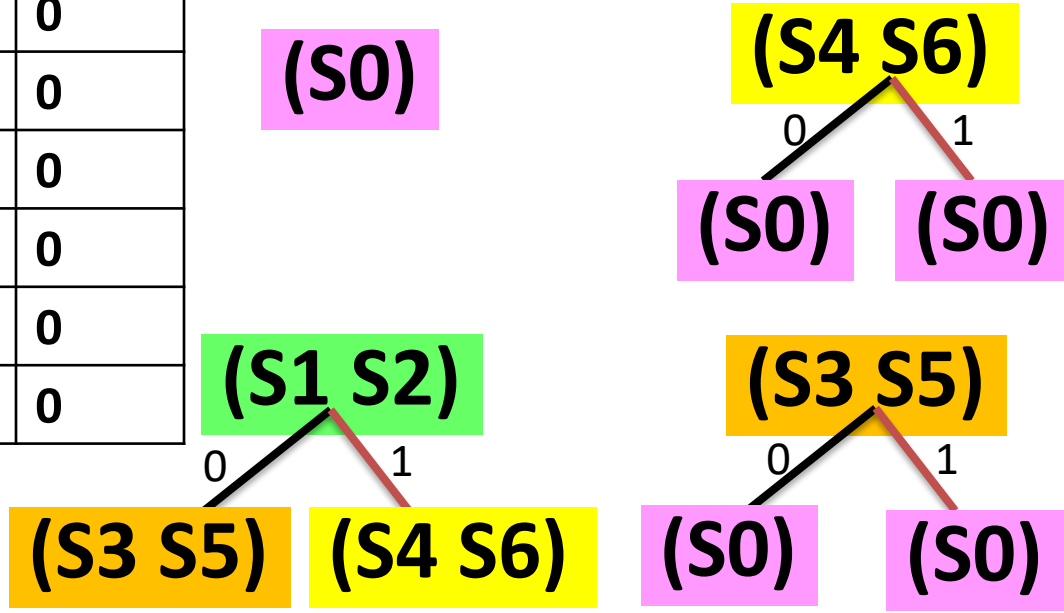
Input	PS	NS		OUTPUT	
		X=0	X=1	X=0	X=1
Reset	S0	S1	S2	0	0
0	S1	S3	S4	0	0
1	S2	S5	S6	0	0
00	S3	S0	S0	0	0
01	S4	S0	S0	1	0
10	S5	S0	S0	0	0
11	S6	S0	S0	1	0



State Minimization Example

Input	PS	NS		OUTPUT	
		X=0	X=1	X=0	X=1
Reset	S0	S1	S2	0	0
0	S1	S3	S4	0	0
1	S2	S5	S6	0	0
00	S3	S0	S0	0	0
01	S4	S0	S0	1	0
10	S5	S0	S0	0	0
11	S6	S0	S0	1	0

(S0 S1 S2 S3 S4 S5 S6)

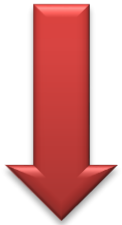


No further partitions possible

State Minimization Example

Input	PS	NS		OUTPUT	
		X=0	X=1	X=0	X=1
Reset	S0	S1	S2	0	0
0	S1	S3	S4	0	0
1	S2	S5	S6	0	0
00	S3	S0	S0	0	0
01	S4	S0	S0	1	0
10	S5	S0	S0	0	0
11	S6	S0	S0	1	0

(S0 S1 S2 S3 S4 S5 S6)



(S0) (S1 S2) (S3 S5) (S4 S6)

(S0) (S1‘) (S3’) (S4’)

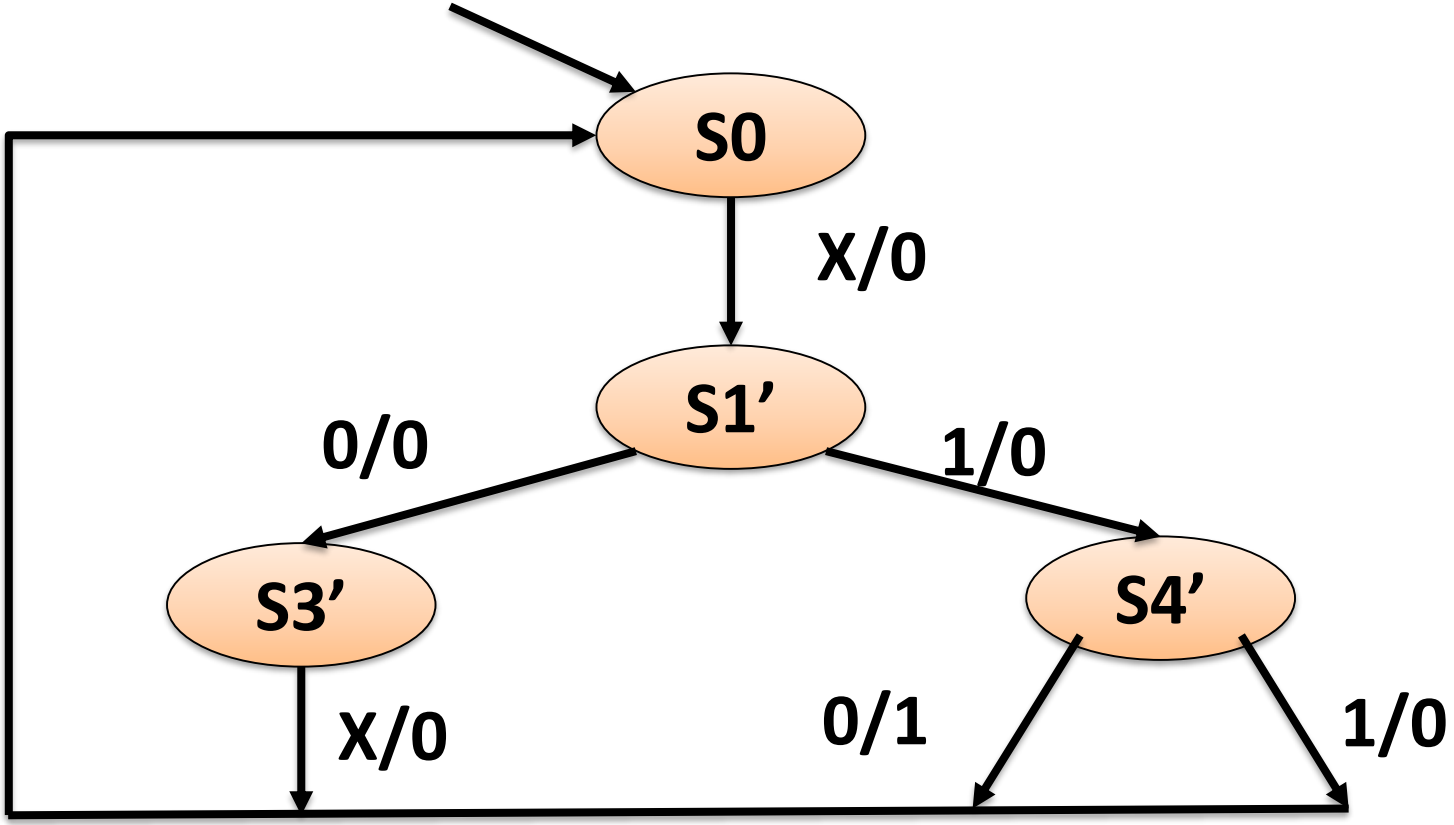
State Minimization Example

Input	PS	NS		OUTPUT	
		X=0	X=1	X=0	X=1
Reset	S0	S1’	S1’	0	0
0	S1’	S3	S4’	0	0
1	S2	S5	S4’	0	0
00	S3	S0	S0	0	0
01	S4’	S0	S0	1	0
10	S5	S0	S0	0	0
11	S4’	S0	S0	1	0

(S0) (S1 S2) (S3 S5) (S4 S6)

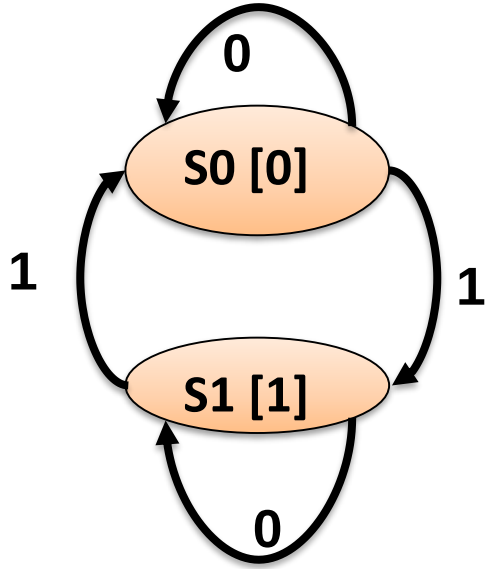
(S0) (S1‘) (S3’) (S4’)

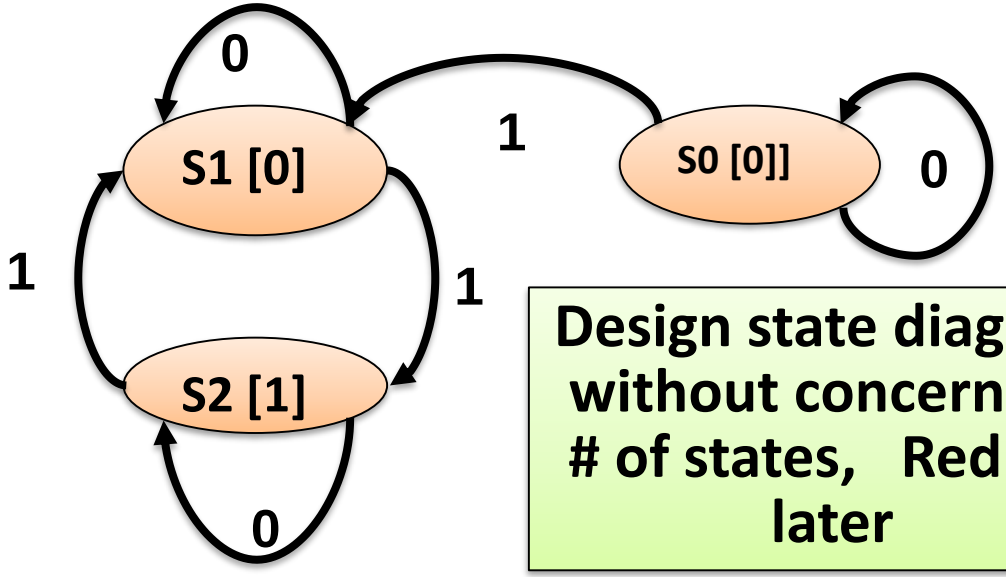
Minimized FSM



Row Matching Method : Fallacies

- Odd Parity Checker:
  - Two alternative state diagrams
  - Identical output behavior on all input strings
  - FSMs are *equivalent*, but require different implementations





Design state diagram without concern for # of states, Reduce later

Partitioning Method

Present State	Next State X=0	Next State X=1	Output
S_0	S_0	S_1	0
S_1	S_1	S_2	1
S_2	S_2	S_1	0

(S0S1S2)
Based on output
(S0S2) (S1)

Partitioning Methd

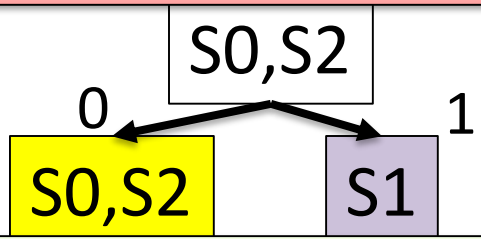
Present State	Next State X=0	Next State X=1	Output
S_0	S_0	S_1	0
S_1	S_1	S_2	1
S_2	S_2	S_1	0

(S0S1S2)

Based on output

(S0S2) **(S1)**

Was not possible with Row matching method



S0,S2 are same

{S0 S2}, {S1} => {S0'}, {S1}

But possible with Partitioning method

Some Definitions

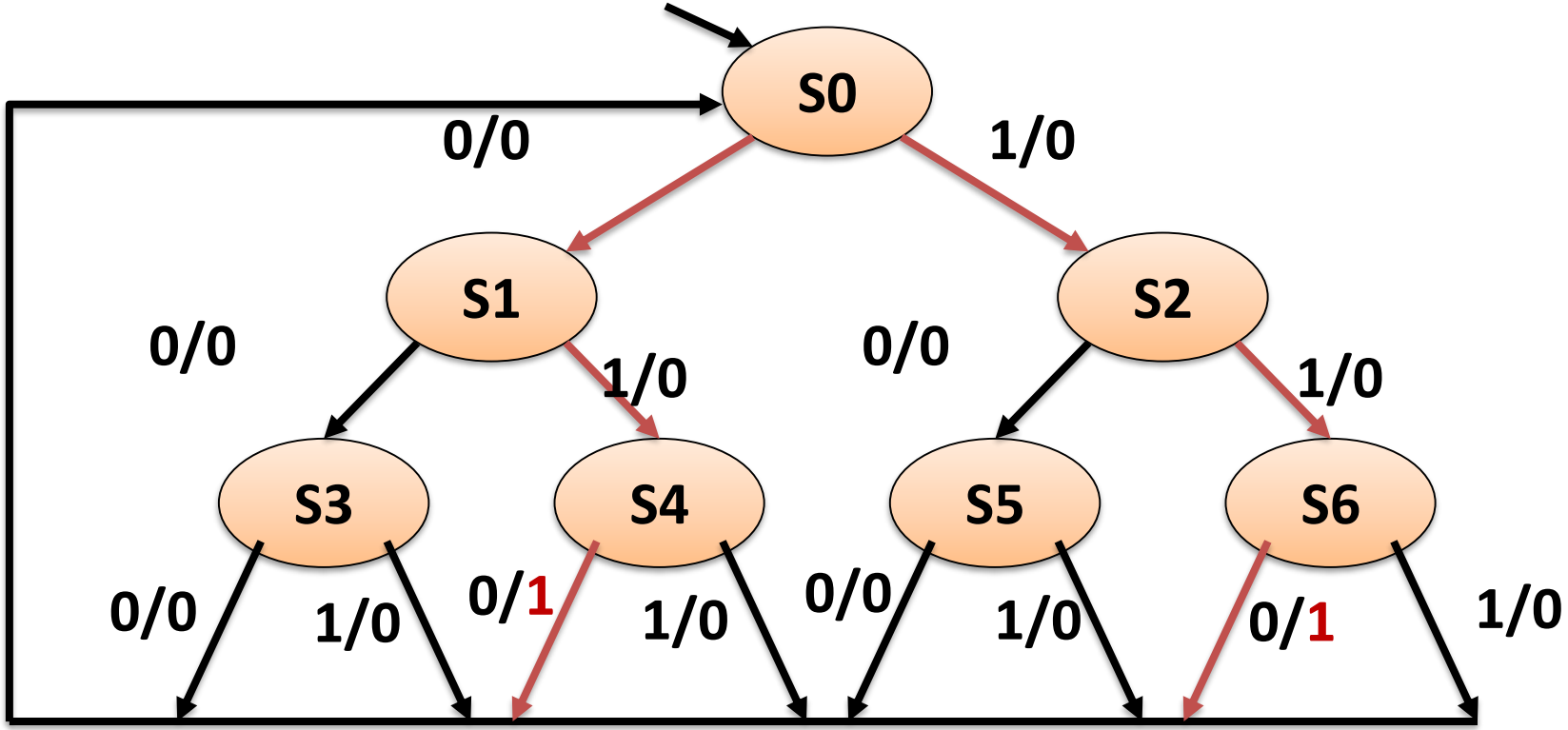
- **State Equivalence**: S1 and S2 are equivalent if for every input sequence applied to machine goes to same NS and Output
  - If S1(t+1)=S2(t+1) and Z1=Z2 then S1=S2

- **Distinguishable States**: Two states S1 and S2 are Distinguishable ***if and only if*** there exist at least one finite input sequence which produce different outputs from S1 and S2

Implication Chart Methods

Sequence Detector Example

- Sequence Detector for 010 or 110
- After is asserted after each 3 bit input sequence if it consist of 110 or 010



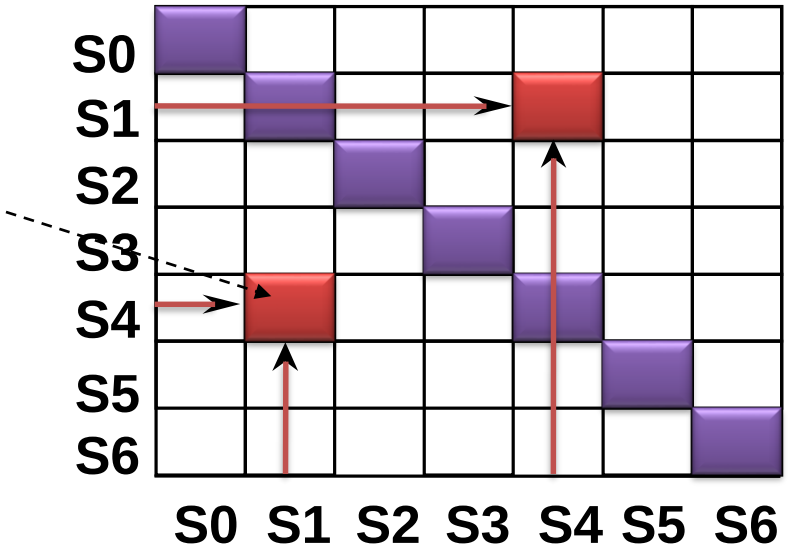
FSM :Tabular Form

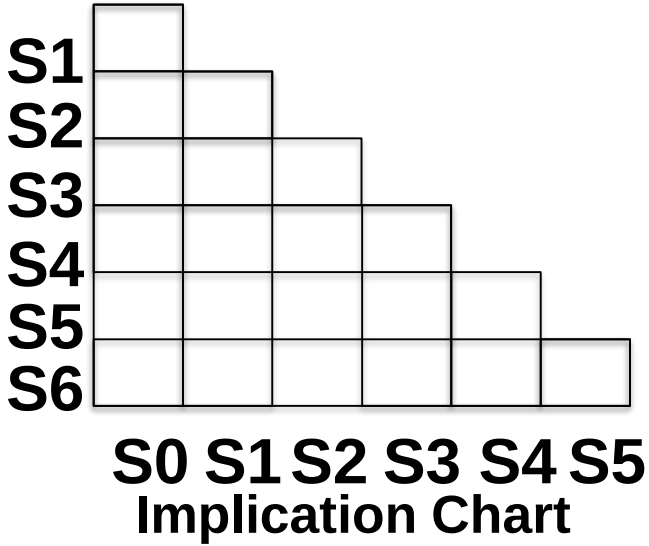
Input	PS	NS		OUTPUT	
		X=0	X=1	X=0	X=1
Reset	S0	S1	S2	0	0
0	S1	S3	S4	0	0
1	S2	S5	S6	0	0
00	S3	S0	S0	0	0
01	S4	S0	S0	1	0
10	S5	S0	S0	0	0
11	S6	S0	S0	1	0

Implication Chart Method

Enumerate all possible combinations of states taken two at a time

Next States Under all Input Combinations





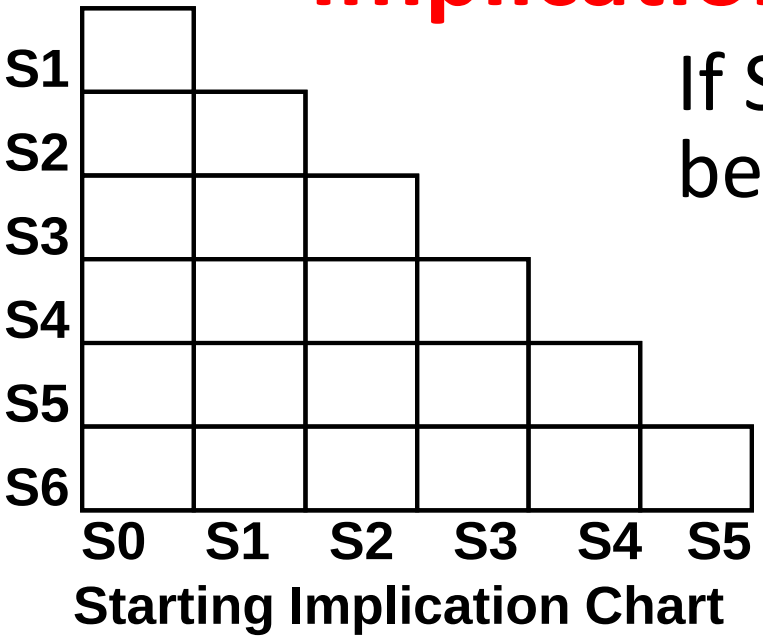
Naive Data Structure: X_{ij} will be the same as X_{ji}
Also, can eliminate the diagonal

Implication Chart Method

Filling in the Implication Chart

- Entry $\mathbf{X_{ij}}$ — Row is $\mathbf{S_i}$, Column is $\mathbf{S_j}$
- $\mathbf{S_i}$ is equivalent to $\mathbf{S_j}$ if outputs are the same and next states are equivalent
- $\mathbf{X_{ij}}$ contains the next states of $\mathbf{S_i}$, $\mathbf{S_j}$ which must be equivalent if S_i and S_j are equivalent
- If $\mathbf{S_i}$, $\mathbf{S_j}$ have different output behavior, then $\mathbf{X_{ij}}$ is crossed out

Implication Chart Method

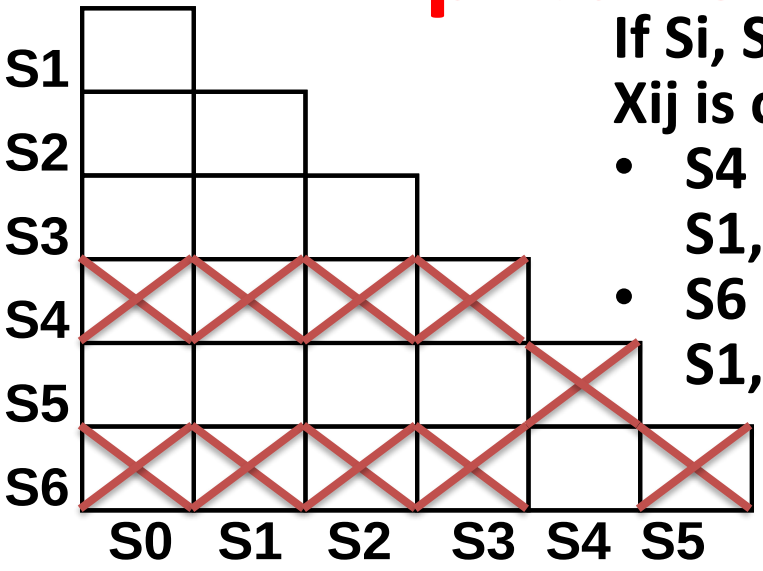


Starting Implication Chart

If S_i, S_j have different output behavior, then X_{ij} is crossed out

Input	PS	NS		OUTPUT	
		X=0	X=1	X=0	X=1
Reset	S0	S1	S2	0	0
0	S1	S3	S4	0	0
1	S2	S5	S6	0	0
00	S3	S0	S0	0	0
01	S4	S0	S0	1	0
10	S5	S0	S0	0	0
11	S6	S0	S0	1	0

Implication Chart Method



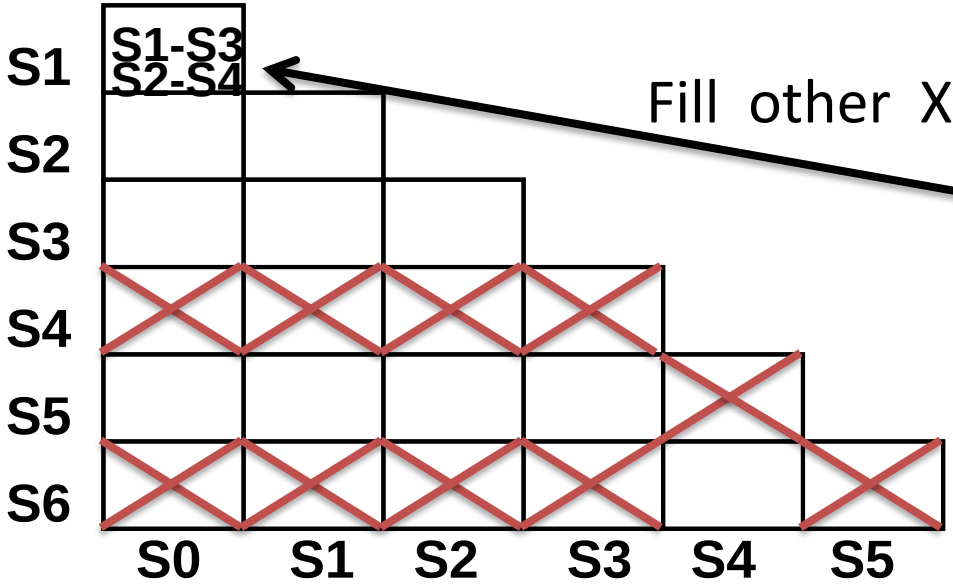
Starting Implication Chart

If Si, Sj have different output behavior, then Xij is crossed out

- **S4 have different output behavior with S0, S1, S2, S3, S5**
- **S6 have different output behavior with S0, S1, S2, S3, S5**

Input	PS	NS		OUTPUT	
		X=0	X=1	X=0	X=1
Reset	S0	S1	S2	0	0
0	S1	S3	S4	0	0
1	S2	S5	S6	0	0
00	S3	S0	S0	0	0
01	S4	S0	S0	1	0
10	S5	S0	S0	0	0
11	S6	S0	S0	1	0

Implication Chart Method



Starting Implication Chart

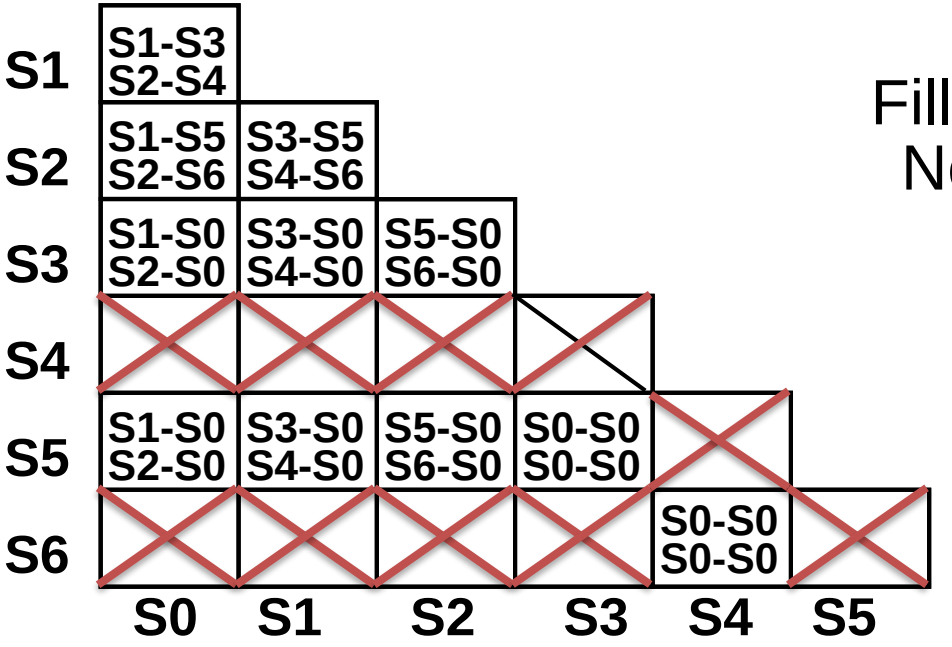
Fill other Xij Based on Next State Transaction

S0 : S1, S2 when X=0, 1

S1 : S3, S4 when X=0,1

Input	PS	NS		OUTPUT	
		X=0	X=1	X=0	X=1
Reset	S0	S1	S2	0	0
0	S1	S3	S4	0	0
1	S2	S5	S6	0	0
00	S3	S0	S0	0	0
01	S4	S0	S0	1	0
10	S5	S0	S0	0	0
11	S6	S0	S0	1	0

Implication Chart Method

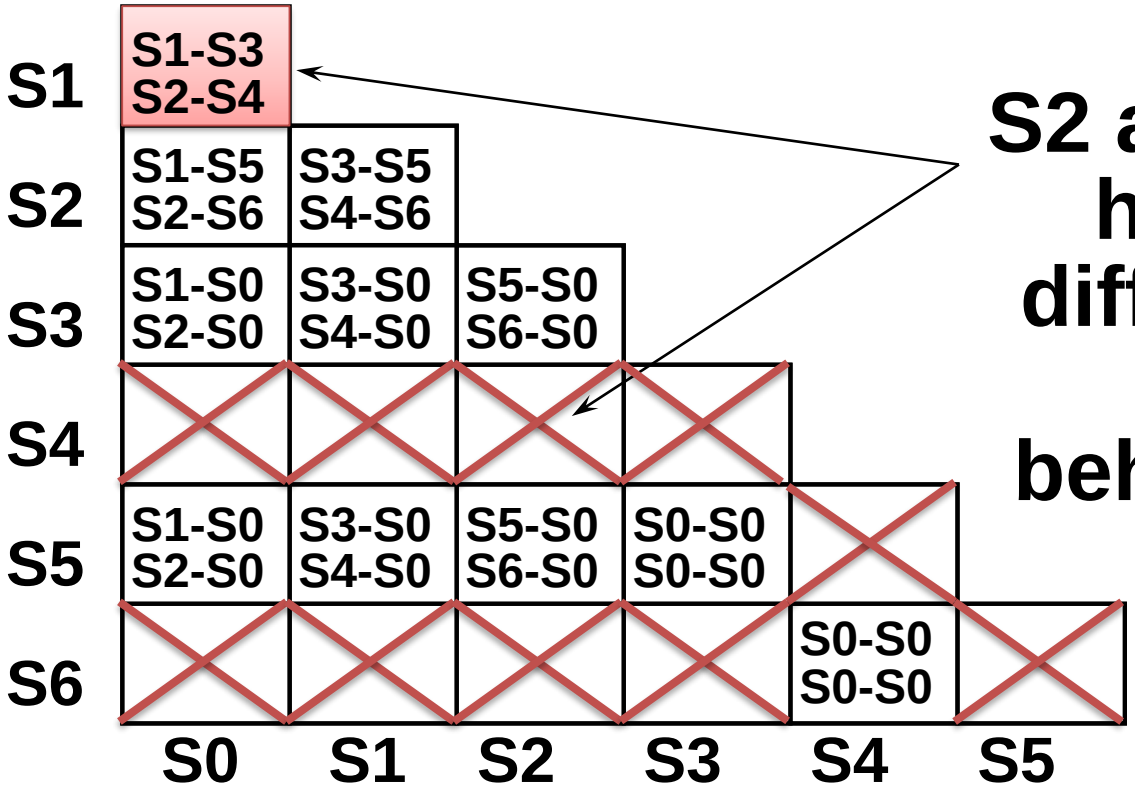


Starting Implication Chart

Fill others Xij Based on Next State Transaction

Input	PS	NS		OUTPUT	
		X=0	X=1	X=0	X=1
Reset	S0	S1	S2	0	0
0	S1	S3	S4	0	0
1	S2	S5	S6	0	0
00	S3	S0	S0	0	0
01	S4	S0	S0	1	0
10	S5	S0	S0	0	0
11	S6	S0	S0	1	0

Implication Chart Method

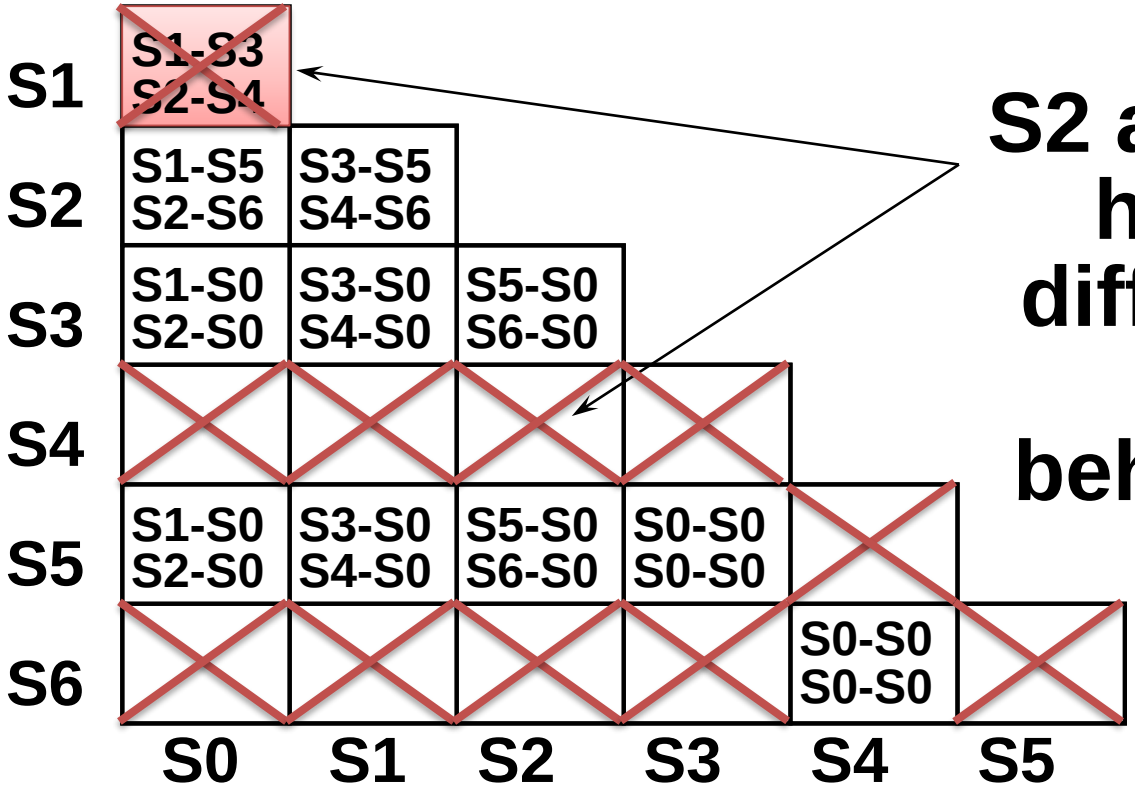


S2 and S4 have different I/O behavior

This implies that S1 and S0 cannot be combined

Marked with Cross

Implication Chart Method

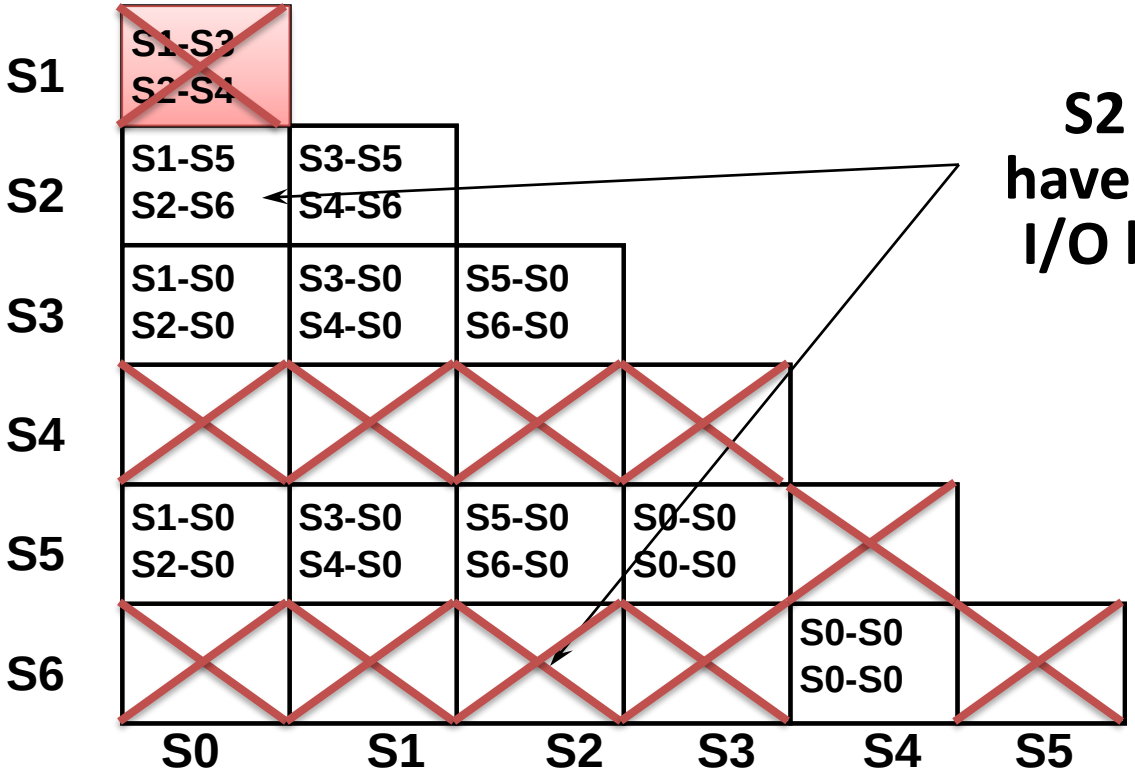


S2 and S4 have different I/O behavior

This implies that S1 and S0 cannot be combined

Marked with Cross

Implication Chart Method

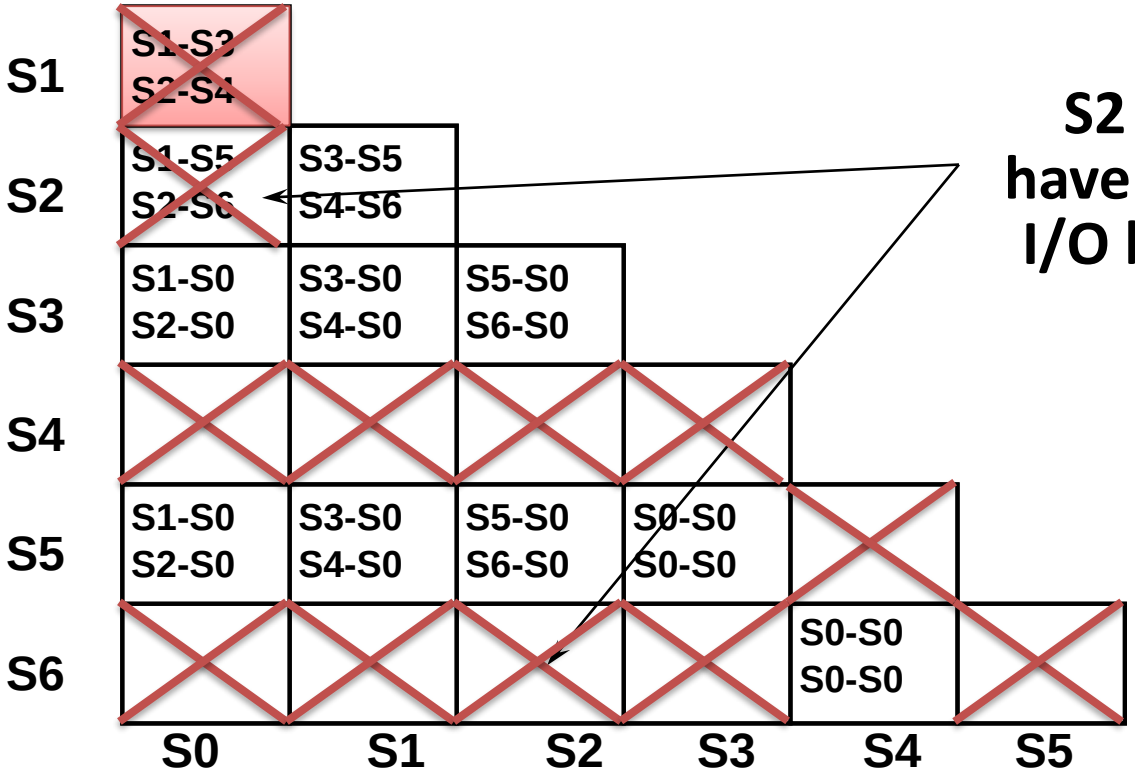


S2 and S6 have different I/O behavior

This implies that S1 and S0 cannot be combined

Marked with Cross

Implication Chart Method

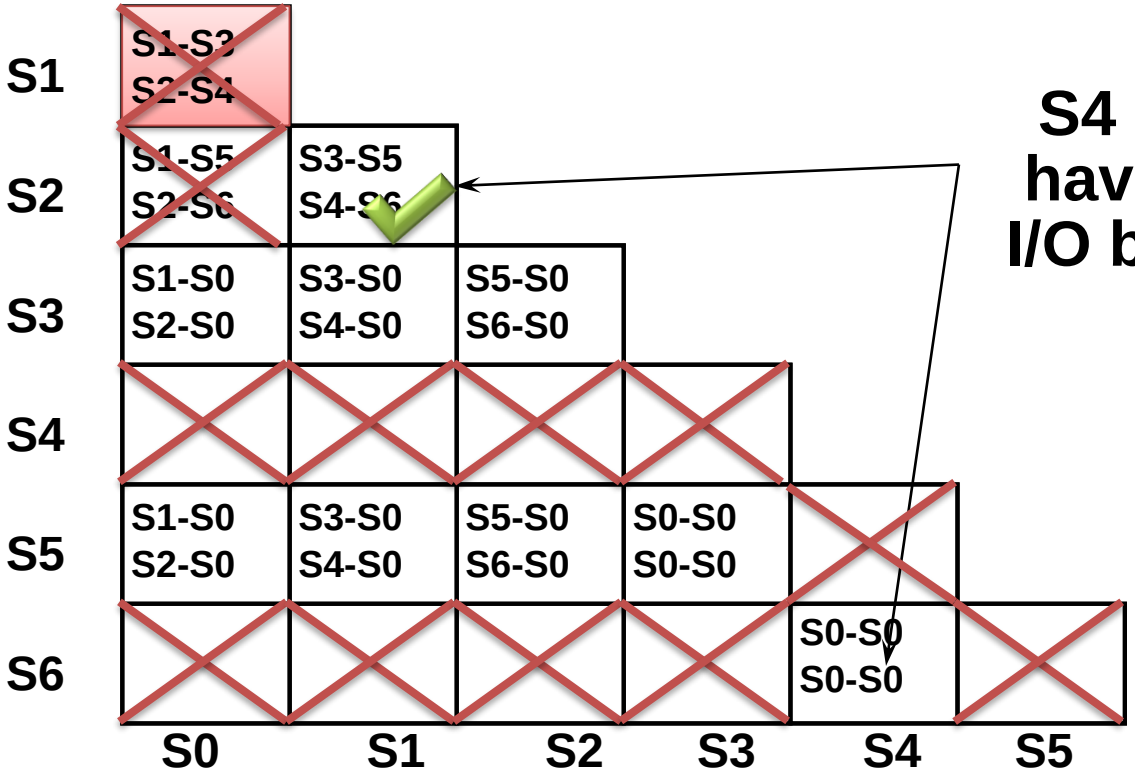


S2 and S6 have different I/O behavior

This implies that S2 and S0 cannot be combined

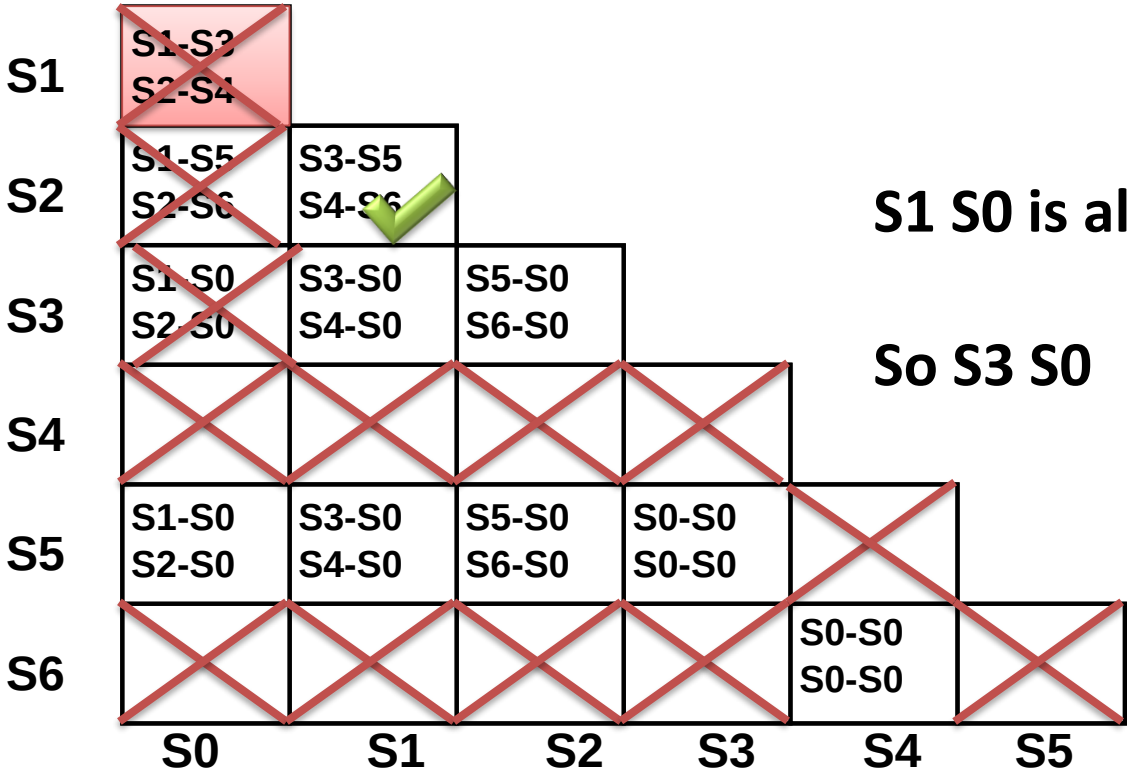
Marked with Cross

Implication Chart Method



S4 and S6 have Same I/O behavior

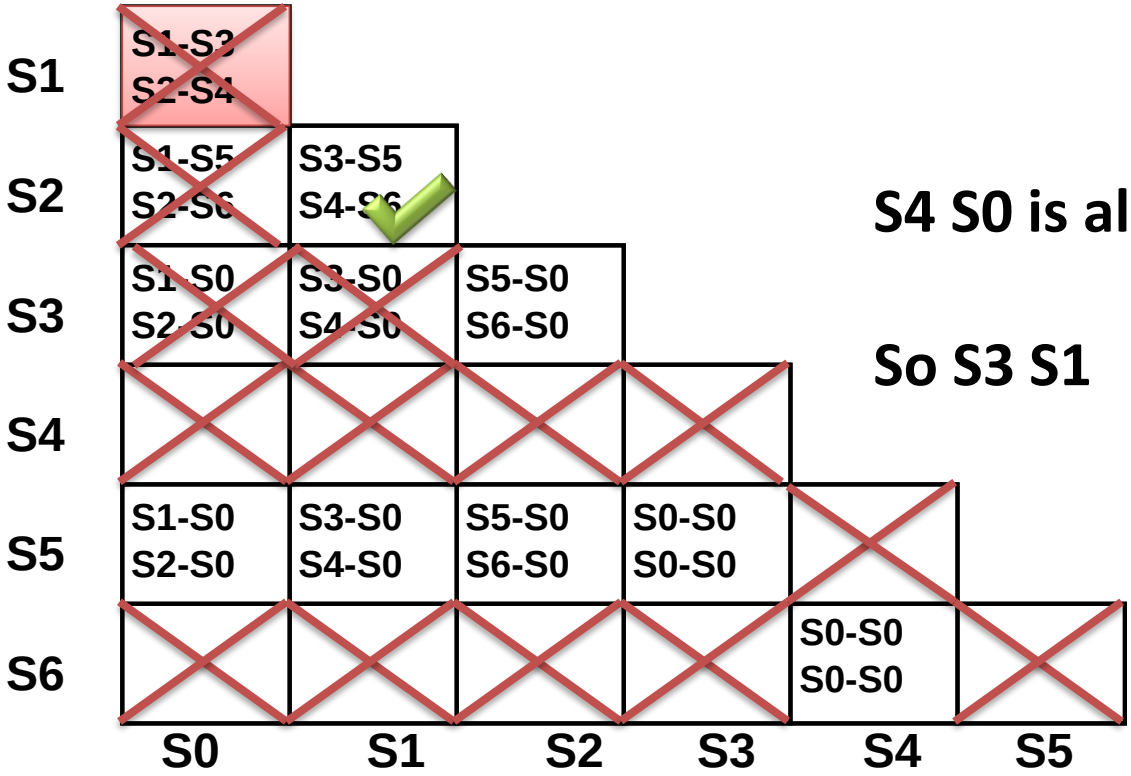
Implication Chart Method



S1 S0 is already crossed

So S3 S0

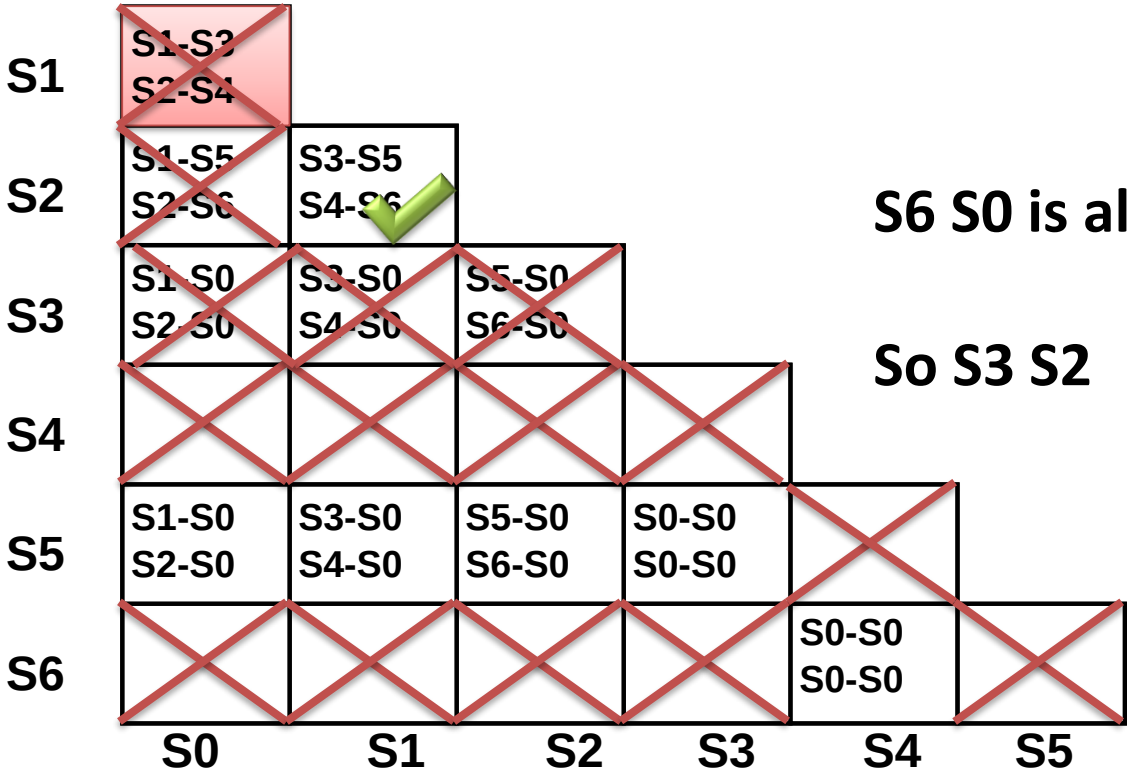
Implication Chart Method



S4 S0 is already crossed

So S3 S1

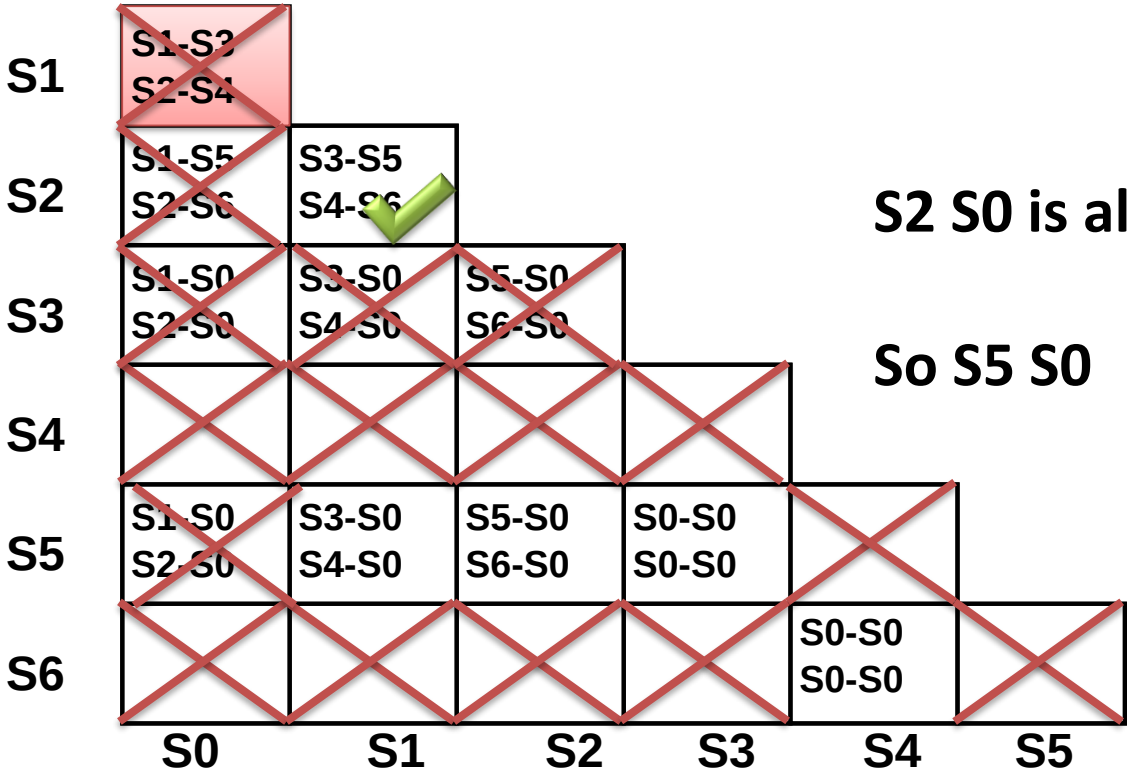
Implication Chart Method



S6 S0 is already crossed

So S3 S2

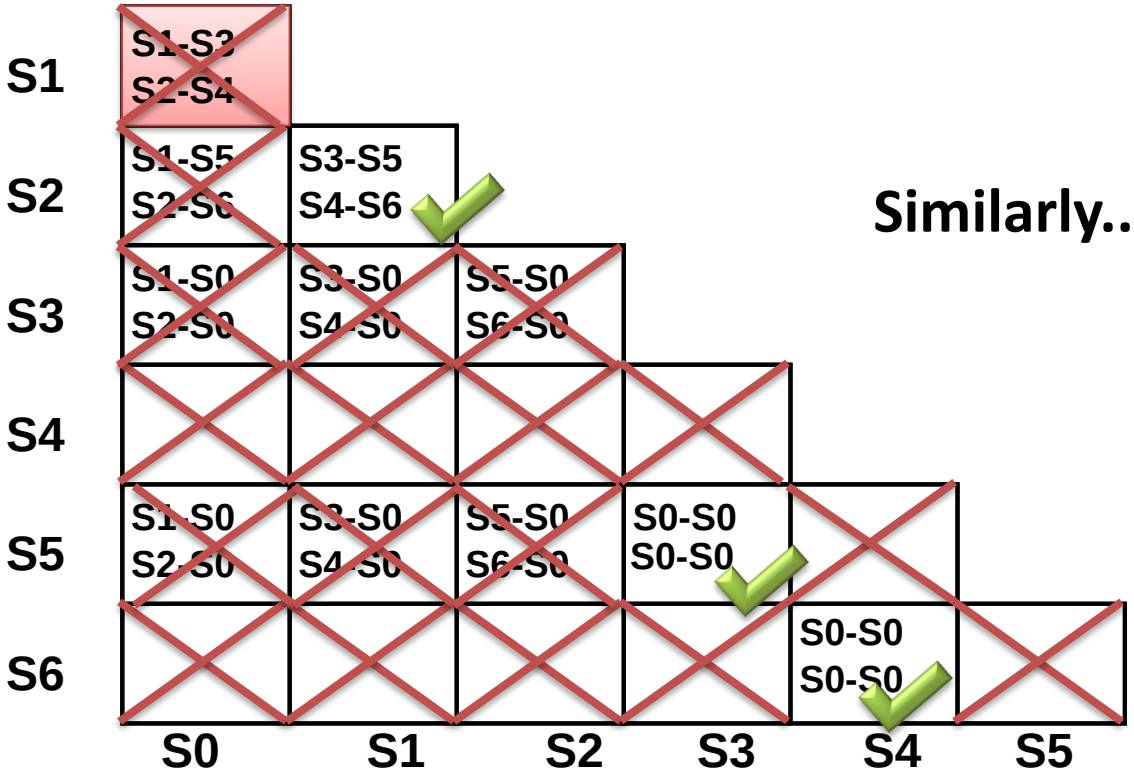
Implication Chart Method



S2 S0 is already crossed

So S5 S0

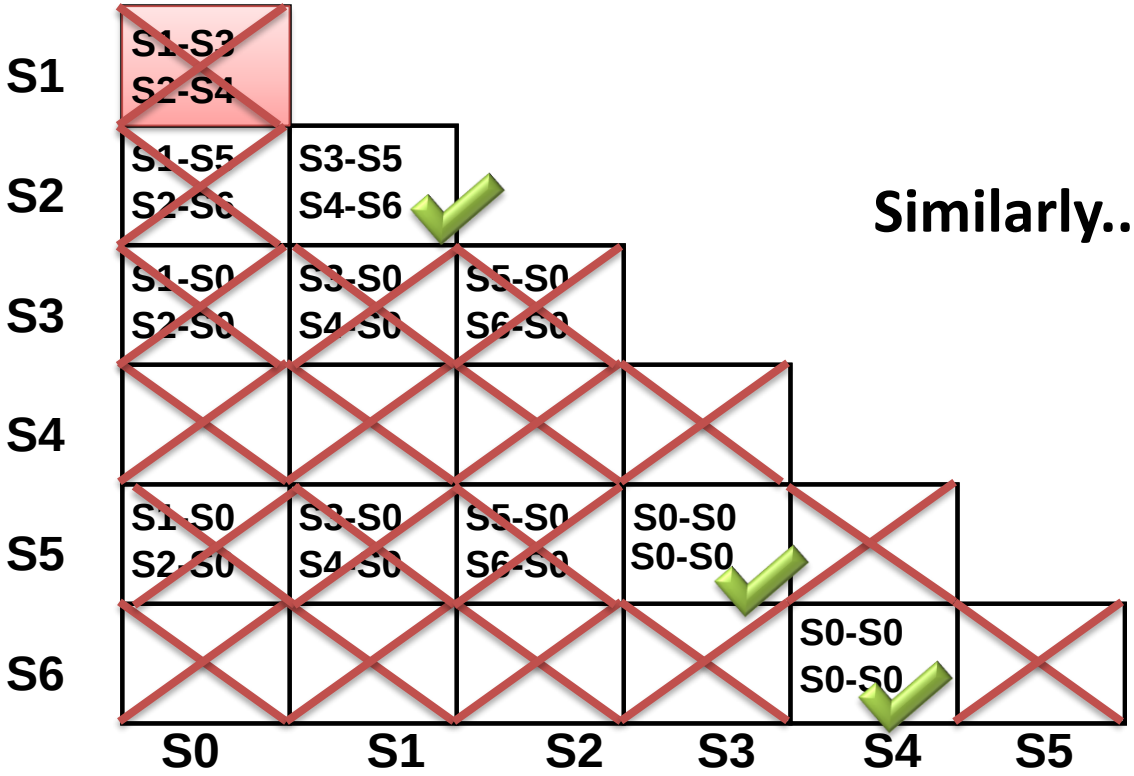
Implication Chart Method



Similarly..

Second Pass Adds No New Information

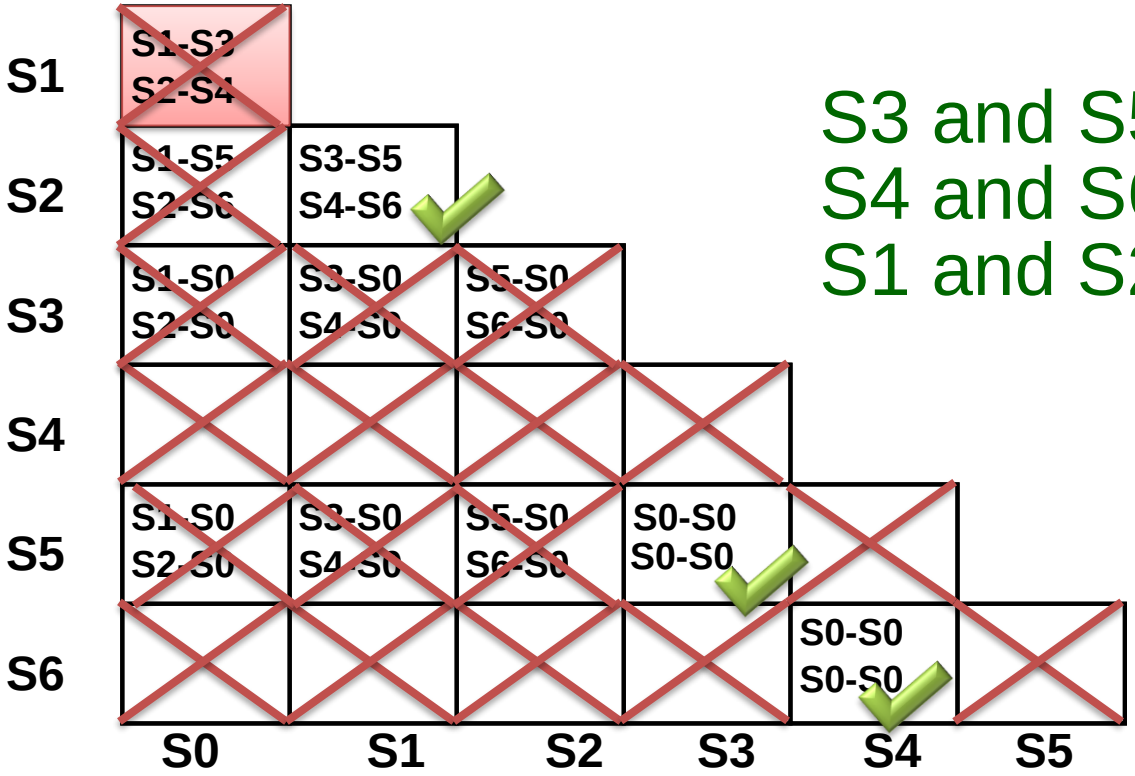
Implication Chart Method



Similarly..

Second Pass Adds No New Information

Implication Chart Method

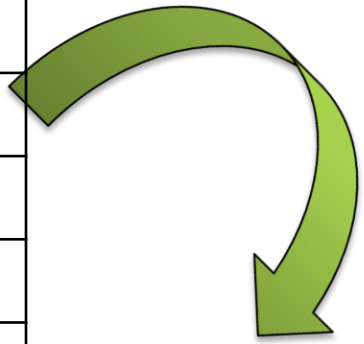


S3 and S5 are equivalent
S4 and S6 are equivalent
S1 and S2 are equivalent

Second Pass Adds No New Information

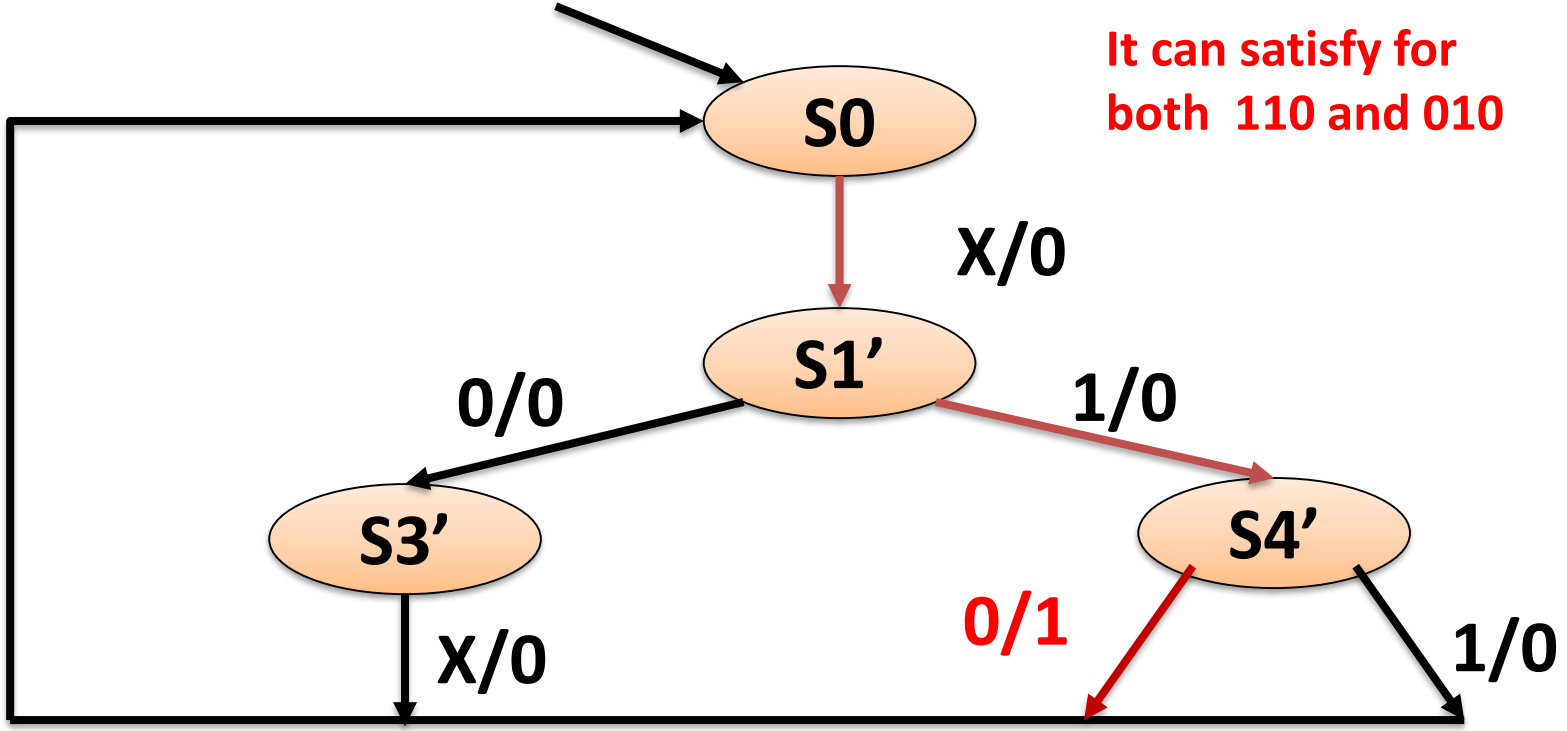
Final : Reduce State Table

Input	PS	NS		OUTPUT	
		X=0	X=1	X=0	X=1
Reset	S0	S1	S2	0	0
0	S1	S3	S4	0	0
1	S2	S5	S6	0	0
00	S3	S0	S0	0	0
01	S4	S0	S0	1	0
10	S5	S0	S0	0	0
11	S6	S0	S0	1	0



Input	PS	NS		Output	
		X=0	X=1	X=0	X=1
Reset	S0	S1’	S1’	0	0
0 or 1	S1’	S3’	S4’	0	0
00 or 10	S3’	S0	S0	0	0
01 or 11	S4’	S0	S0	1	0

Minimized FSM



Outline

- FSM Optimization: State Minimization
  - Row Matching method
  - Partitioning method
  - Implication Chart
- **FSM Optimization : State Encoding**
  - Binary/Sequential, Gray, One-hot
  - Heuristic Based

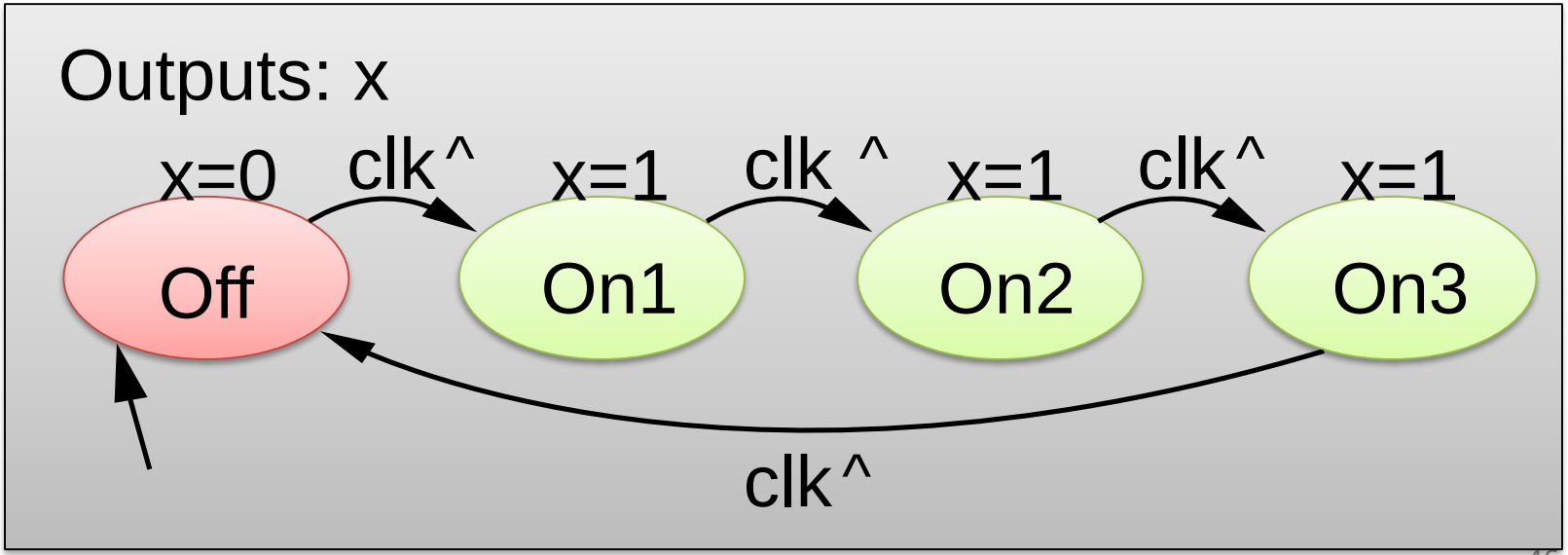
State Encoding/Assignment

State assignment

- State encoding:
  - Assigning unique binary value to each state
  - So that we can implement FSM
- Since we don't care about the actual flip-flop values for each state
- We can assign each state to any binary number we like **as long as**
  - Each state is assigned a unique binary number

Example of FSM: 3 cycle Laser

- Require 4 States
- Assigning unique binary value/code to each state
- Options available : 4!



State Encoding Example

- Four states

S0, S1, S2, S3

- 4! options

SL	S0	S1	S2	S3
1	00	01	10	11
2	00	01	11	10
3	00	10	01	11
4	00	10	11	01
5	00	11	01	10
6	00	11	10	01
7	01	00	11	10
8	01	00	10	11
9	01	10	11	00
10	01	10	00	11
11	01	11	10	00
12	01	11	00	11

SL	S0	S1	S2	S3
13	10	01	00	11
14	10	01	11	00
15	10	00	01	11
16	10	00	11	01
17	10	11	01	00
18	10	11	00	01
19	11	00	01	10
20	11	00	10	01
21	11	10	01	00
22	11	10	00	01
23	11	01	10	00
24	11	01	00	11

State assignment

- Since we don't care about the actual flip-flop values for each state we can assign each state to any binary number we like **as long as** each state is assigned a unique binary number
- Suppose a FSM have 6 states: Modulo 6 Counter
- If we use 3 bits to encode the 6 states, we have

$$\binom{8}{6} = \frac{8!}{6!(8-6)!}$$

possible choosing the 6 states from 8 total state

  - Than encodings

State Encoding

- The cost & delay of FSM implementation depends on encoding of symbolic states.
  - e.g., 4 states can be encoded in 4! = 24 different ways
- There are more than n! different encodings for n states.
  - **Exploration of all encodings is impossible**, therefore heuristics are used
- Heuristics Used
  - One-hot encoding, Minimum-bit change, Prioritized adjacency

State assignment

state	Encoding 1 (binary)	Encoding 2 (Gray)	Encoding 3
a	000	000	000
b	001	001	100
c	010	011	010
d	011	010	101
e	100	110	011

State Encoding

- Binary and Gray encoding use the minimum number of bits for state register
- Gray and Johnson code: Two adjacent codes differ by only one bit
  - Reduce simultaneous switching
  - Reduce crosstalk, Reduce glitch

One-hot encoding

- One flip-flop per state encoding
- Leads to greater number of flip-flops than binary encoding but possibly to simpler logic

State Assignment Problem

- Some state assignments are better than others.
- The state assignment influences the complexity of the state machine.
  - The combinational logic required in the state machine design is dependent on the state assignment.
- Types of state assignment
  - Binary encoding: 2^N states → N Flip-Flops
  - Gray-code encoding: 2^N states → N Flip-Flops
  - One-hot encoding: N states → N Flip-Flops

FSM: State Assignment

Example

Design a FSM that detects a sequence of two or more consecutive ones on an input bit stream.

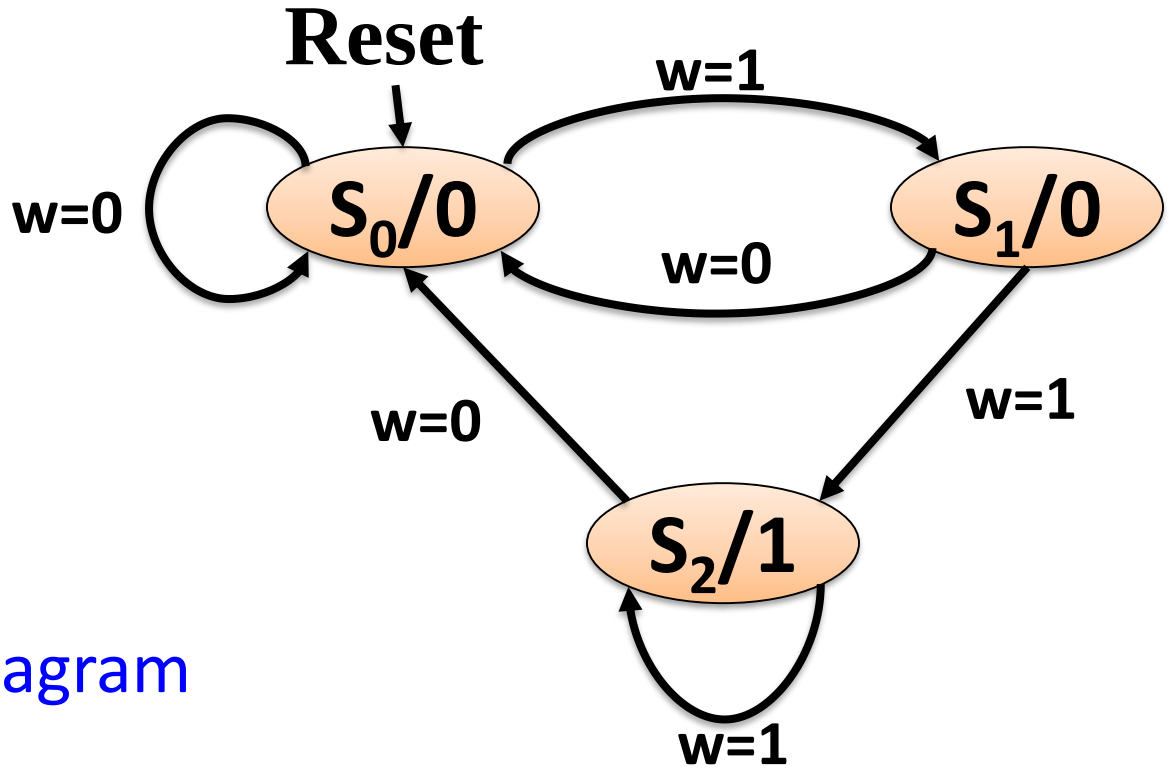
The FSM should output a 1 when the sequence is detected, and a 0 otherwise.

FSM: State Assignment

Input: 0 1 1 1 0 1 0 1 1 0 1 1 1 0 1 …

Output: 0 0 1 1 0 0 0 0 1 0 0 1 1 0 0 …

FSM: State Assignment



State Diagram

FSM: State Assignment

Present State	Next State		Output
	w = 0	w = 1	
S_0	S_0	S_1	0
S_1	S_0	S_2	0
S_2	S_0	S_2	1

State Table

FSM: State Assignment #1

State Assigned Table

Present State			Next State						Output
			w = 0			w = 1			
	Q_A	Q_B		Q_A^+	Q_B^+		Q_A^+	Q_B^+	z
S_0	0	0	S_0	0	0	S_1	0	1	0
S_1	0	1	S_0	0	0	S_2	1	0	0
S_2	1	0	S_0	0	0	S_2	1	0	1
	1	1		d	d		d	d	d

Using Binary Encoding for the State Assignment

$D_B = wQ'_AQ_B'$

$D_A = w(Q_A + Q_B)$

$z = Q$

FSM: State Assignment #2

State Assigned Table

Present State			Next State						Output
			w = 0			w = 1			
	Q_A	Q_B		Q_A^+	Q_B^+		Q_A^+	Q_B^+	z
S_0	0	0	S_0	0	0	S_1	0	1	0
S_1	0	1	S_0	0	0	S_2	1	1	0
S_2	1	1	S_0	0	0	S_2	1	1	1
	1	0		d	d		d	d	d

Using Gray-code Encoding for the State Assignment

$D_A = w.Q_B$

$D_B = W$

$Z = Q_A$

FSM: State Assignment #3

State Assigned Table

Present State				Next State							
				w = 0				w = 1			
	Q_A	Q_B	Q_C		Q_A^+	Q_B^+	Q_C^+		Q_A^+	Q_B^+	Q_C^+
S_0	0	0	1	S_0	0	0	1	S_1	0	1	0
S_1	0	1	0	S_0	0	0	1	S_2	1	0	0
S_2	1	0	0	S_0	0	0	1	S_2	1	0	0

Using One-hot Encoding for the State Assignment

For each state only one flip-flop is set to 1.

The remaining combination of state

$D_A = w.Q_C'$

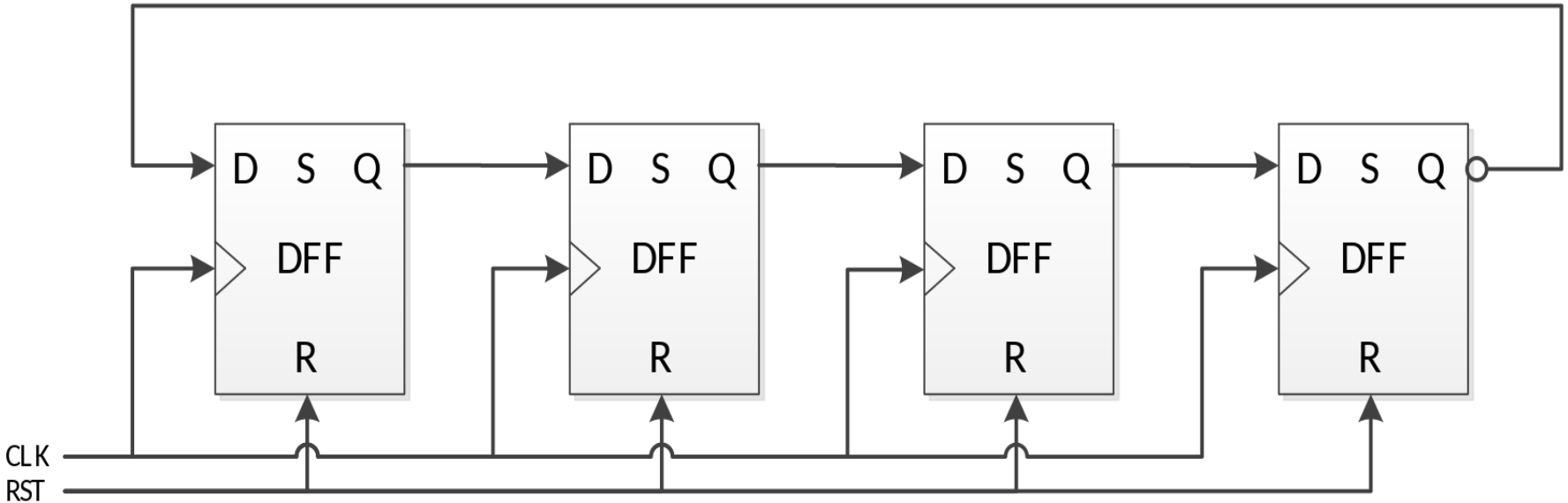
$D_B = w.Q_C$

$D_C = w'$

State Encoding

#	Binary	Gray	Johnson	One-hot
0	000	000	0000	00000001
1	001	001	0001	00000010
2	010	011	0011	00000100
3	011	010	0111	00001000
4	100	110	1111	00010000
5	101	111	1110	00100000
6	110	101	1100	01000000
7	111	100	1000	10000000

Johnson Counter



Thanks